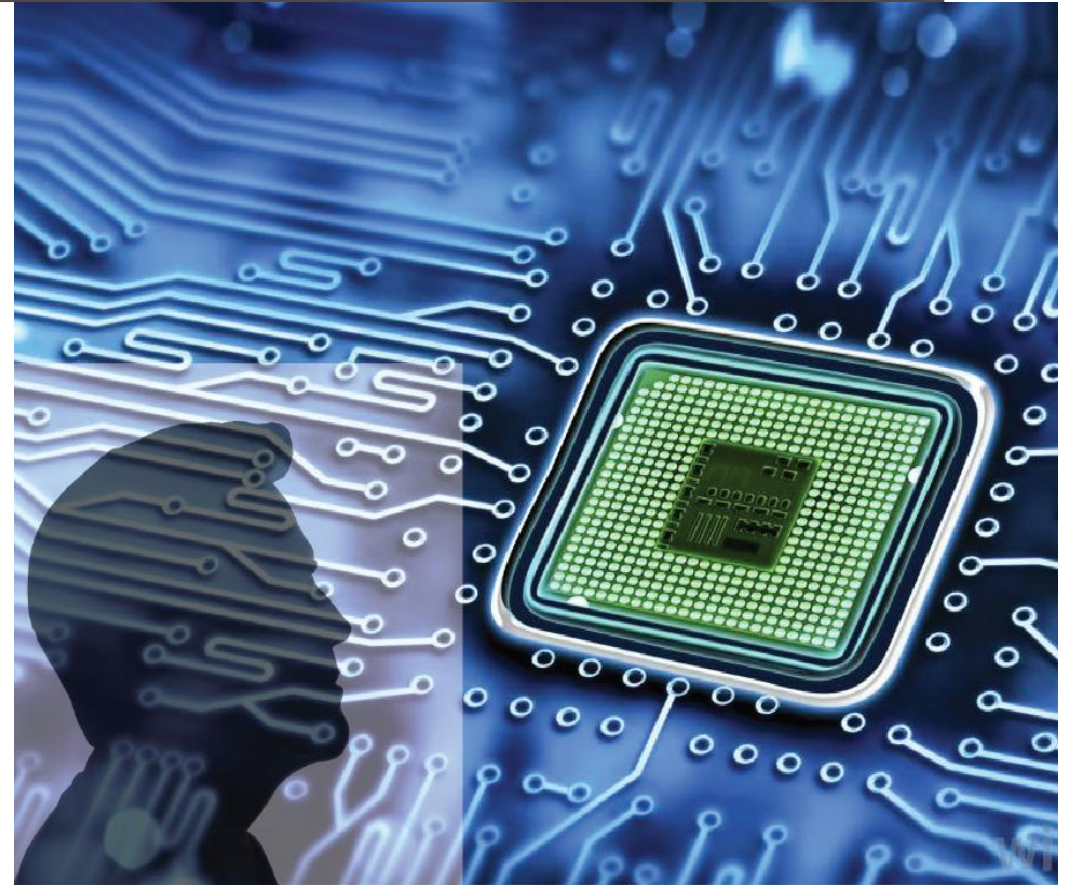


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-6 Lecture-1 High Level Synthesis

Sneh Saurabh
16th September, 2025

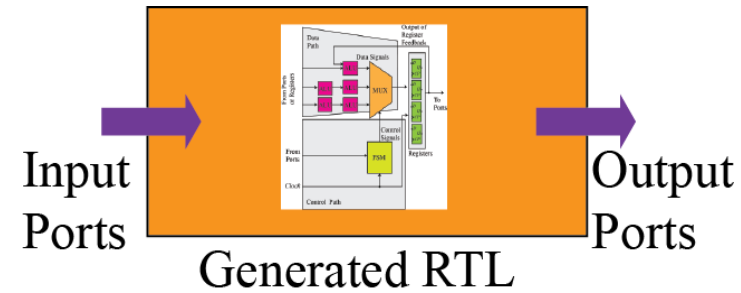
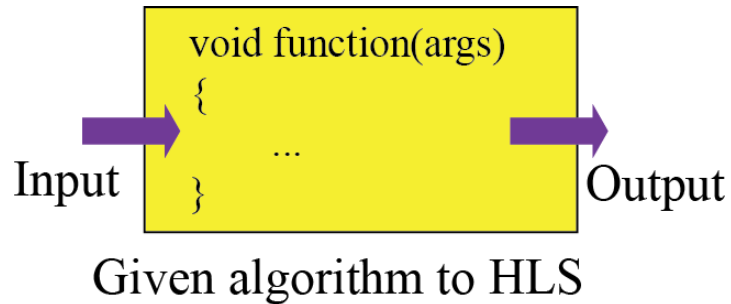


High-level Synthesis



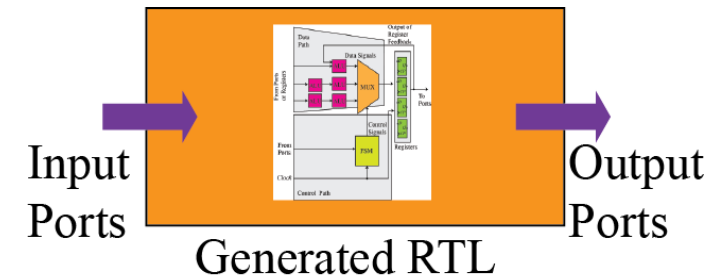
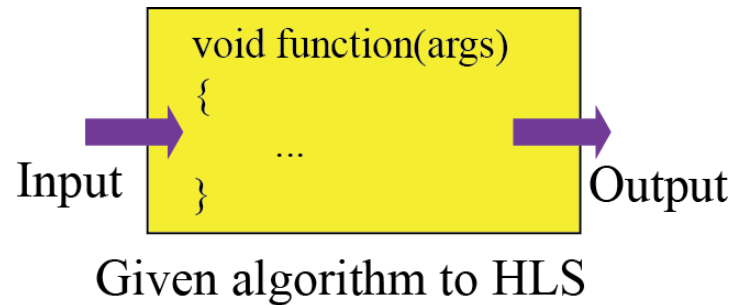
Problem
Definition

HLS: Mapping IOs



- Given an input, HLS must preserve the relation between input and output
 - HLS is free to rearrange/adjust the internal computation
 - Need to create a mapping between input and outputs of the two models
-
- Direct Mapping sometimes possible
 - Example: SystemC models with `sc_in` and `sc_out` is used in the algorithmic description
 - For pure C/C++ high-level models, infer using arguments given to the function
 - Can be automatic
 - Sometimes user input might be needed for defining cycle-based semantics (clock is easy to model in SystemC)

HLS: How to Map Behavior?

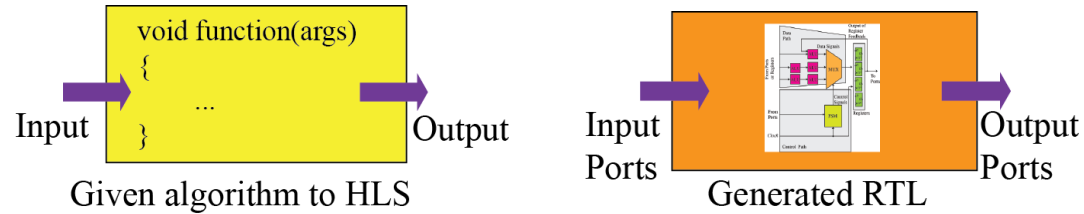


- Given an input, HLS must preserve the relation between input and output
 - What behavior needs to be preserved?

Three kinds of Mapping

1. Cycle accurate
2. Cycle delayed
3. Latency insensitive

HLS: IO Cycle Accurate

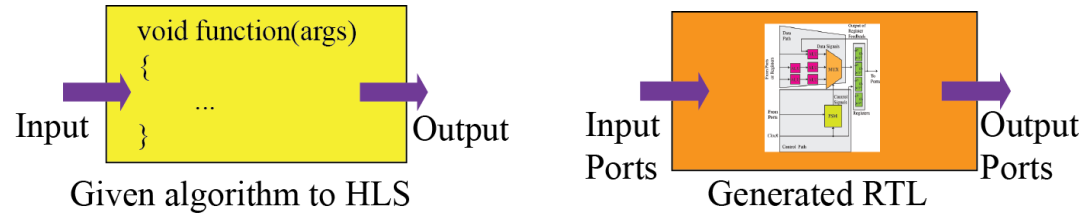


Cycle-accuracy

- Assume that inputs have same values at each cycle boundary for the two models
 - Ensure that the outputs have equivalent values in the two models at each cycle boundary
- Timing of IO operations is fixed
- Memory access can be considered as IO:
 - Access of memory should be in the same cycle in the two models

- Freedom for HLS tool:
 - Sharing is abstracted and can be optimized
 - Internal scheduling can be chosen
 - FSM implementation
- Restrictions for HLS tool:
 - Scheduling is well-defined for the IOs
 - Automatic pipelining cannot be done
- Verification becomes easier
 - Either model can be used in simulation
- Equivalence checking becomes easier

HLS: IO Cycle Delayed



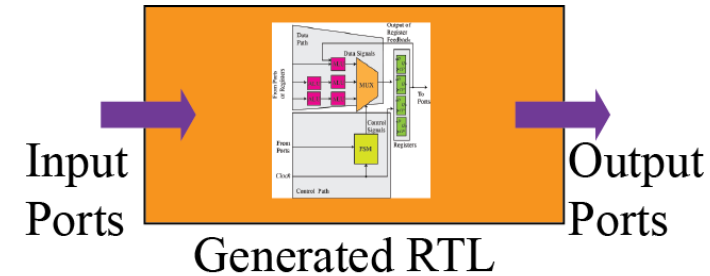
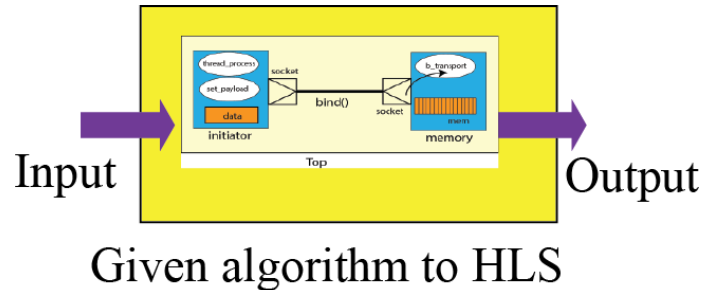
Cycle delayed

- Assume that inputs have same values at each cycle boundary for the two models
 - Ensure that the outputs of the RTL is equivalent to the delayed version of the output produced by the high-level model

- Latency is relaxed
- Pipelining is allowed
 - $h(n) = r(n + d)$ where n is the cycle count and d is the pipeline depth

- Pipeline depth is determined by the HLS tool
 - Designer can constrain it
- Verification becomes challenging
 - Pipeline depth can be different for different outputs (verification complicates further)

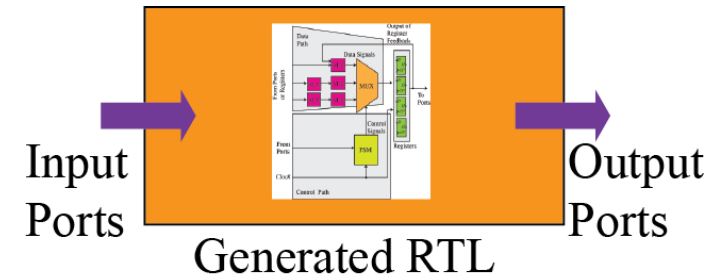
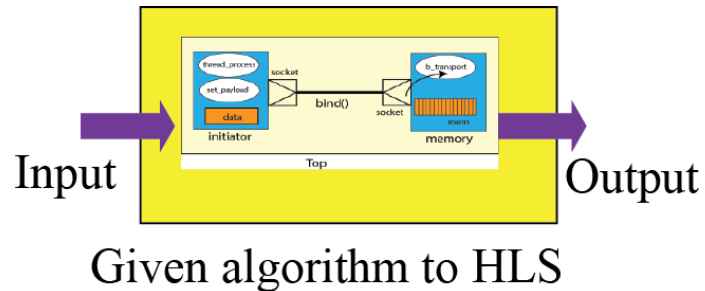
HLS: IO Latency-insensitive (1)



Latency-insensitive designs

- Data streamed between two design components, and a handshaking signal controls the operation
 - Each computational component produces/consumes data tokens
-
- Input to the HLS tools should contain distinct description of data processing components and protocol
 - Data processing is latency-insensitive (behavior does not change if latency is changed)
 - Protocol is interpreted in cycle accurate manner

HLS: IO Latency-insensitive (2)



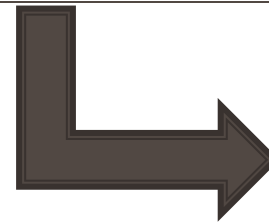
- HLS needs to ensure that for the same stream of input tokens, the two models should produce equivalent tokens at each output

- Scope of optimization increases
 - No restriction on the timing of each token at the output
 - No restriction on the order of tokens produced at the output
 - Care should be taken that avoids resources sitting idle

- Verification should consider latency insensitivity

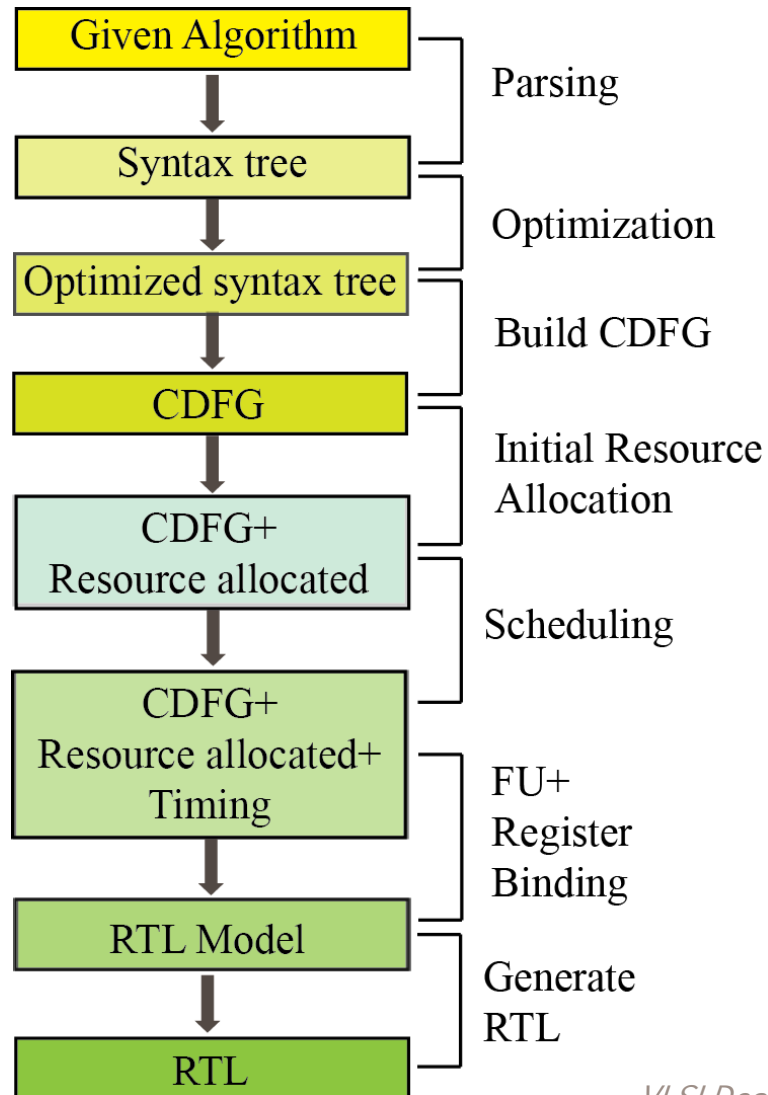
- TLM naturally leads to latency insensitive designs
 - TLM can be refined to make cycle-accurate or cycle-delayed

High-level Synthesis



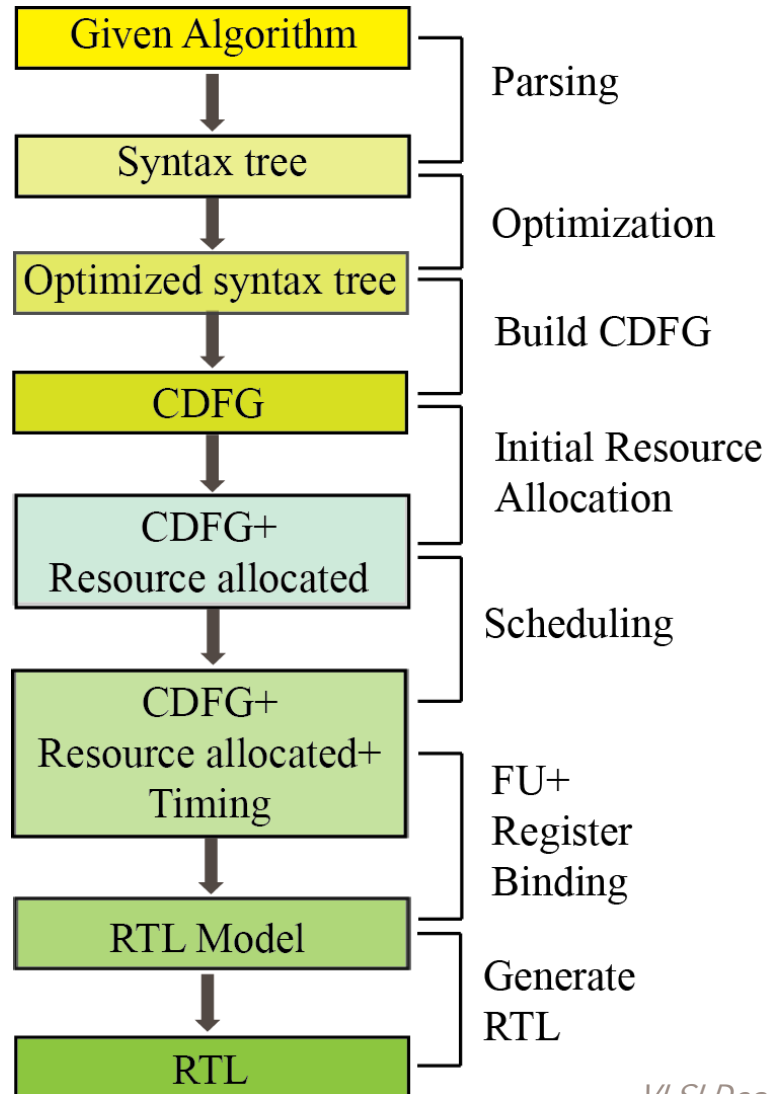
Flow

HLS: Typical Flow



▪ We will discuss each task in this lecture.

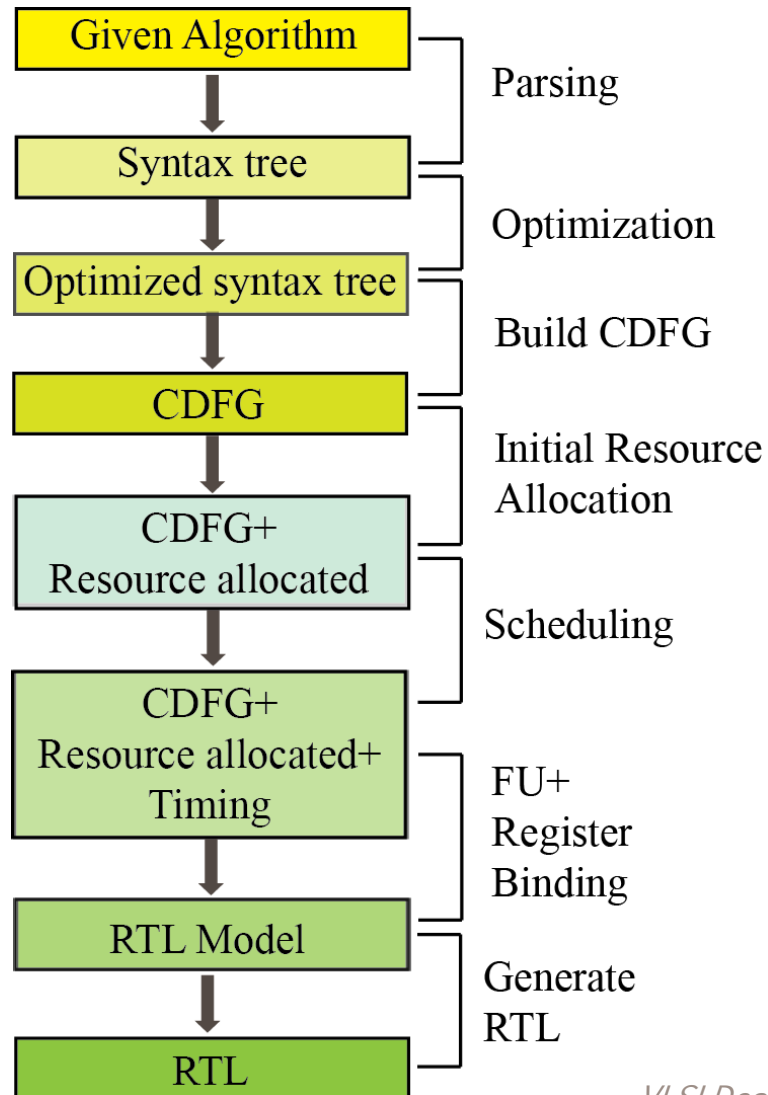
HLS: Parsing



- Reads the given algorithm (C, C++, MATLAB, ... files)

- Lexical analysis: keywords, identifiers (break text into tokens)
- Grammar/syntax checking
- Syntax tree built:
 - If grammar is correct
 - Hierarchical data structure

HLS: Optimization



- Traditional compiler optimizations applied
- These optimizations can be performed on the parse tree or after CDFG is built

Compiler Optimizations: Examples

Constant Propagation: replace expression with constant when possible

```
a = 8*8;  
b = (a*1024)/32;  
c = (b+32+b+32);
```

```
a = 64;  
b = 2048;  
c = 4160;
```

Common subexpression elimination: replace identical subexpression in multiple expressions with a single variable if it reduces the cost, such as area

```
x = p+a*b;  
y = q+a*b;
```

```
c = a*b;  
x = p+c;  
y = q+c;
```

Strength reduction: replace an expensive arithmetic operation with an equivalent less expensive operation

```
x = a*64;  
y = b/4;  
z = c*17;
```

```
x = a<<6;  
y = b>>2;  
z = (c<<4)+c;
```

Compiler Optimizations: Examples

Dead code elimination:

- Remove portion of a code that will never be executed
- Remove the portion of a code whose result is never used.

```
debug = 0;  
if (debug == 1) { // dead code  
    x = a; // dead code  
} // dead code
```

Copy propagation:

- Use a copy assignment of the form $x = a$ for optimizing

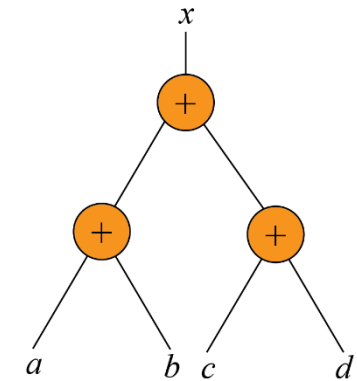
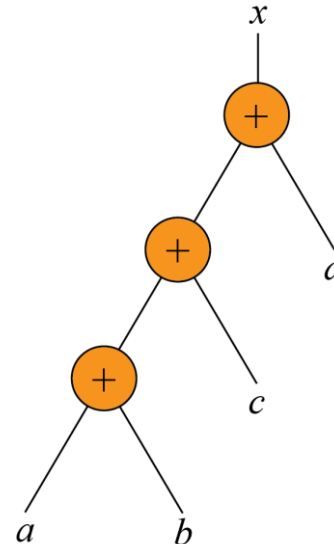
```
x = a;  
y = x+5;  
z = x+9;
```

```
x = a;  
y = a+5;  
z = a+9;
```

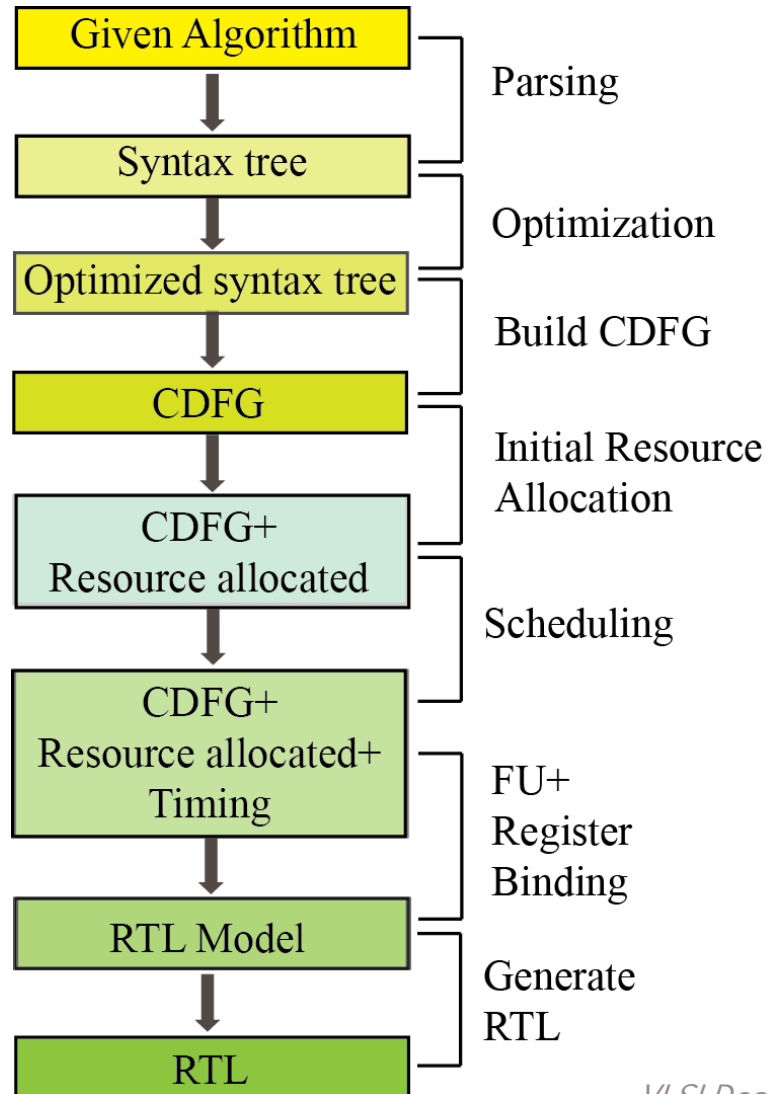
```
y = a+5;  
z = a+9;
```

Tree height reduction:

- Restructure the tree such that the height gets reduced.
- The maximum delay in computation gets reduced



HLS: Build CDFG



- Control Data Flow Graph (CDFG): data structure to represent the control flow (if-then, branches, etc.) and data dependencies between operations

CDFG consists of:

1. **Control Flow Graph:** represents branching points (decisions like if-else, loops, etc.) that control the flow of algorithm
2. **Data Flow Graph:** represents dependencies among data and operations

Motivation to create CDFG:

- Encode the behavior of the program (language independent)
- Facilitate analysis: several known graph algorithms can be helpful
- Facilitate optimization: compiler optimizations and many more

Control Flow Graph (CFG)

- CFG can be built in many ways
 - Depending on the analysis and operations that need to be carried out

- CFG can be defined using **basic blocks**

Basic Blocks

- Grouping of sequential operations without branching
 - Flow of control can only enter at the beginning and leave at the end
- Only the last statement of a basic block can be a branch statement
- Only the first statement of a basic block can be a target of a branch

Basic Blocks

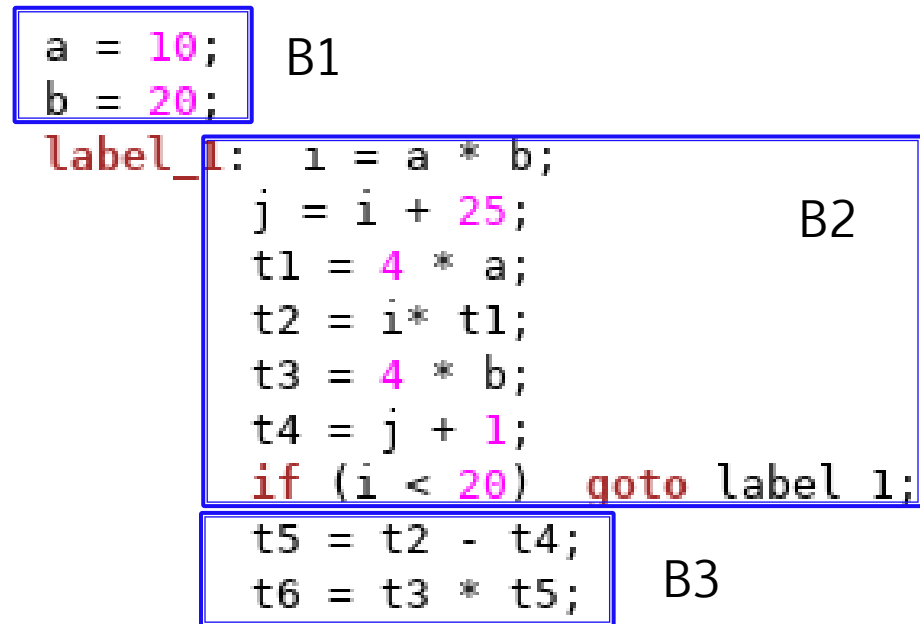
How to identify basic blocks?

- Each basic block begins with a **leader statement** operations
- Identify leader statements by using the following rules:
 - The first statement in the program is a leader
 - Any statement that is the target of a branch statement is a leader
 - Any statement that immediately follows a branch or return statement is a leader
- Statements starting with leader statement to the next leader statement form a basic block

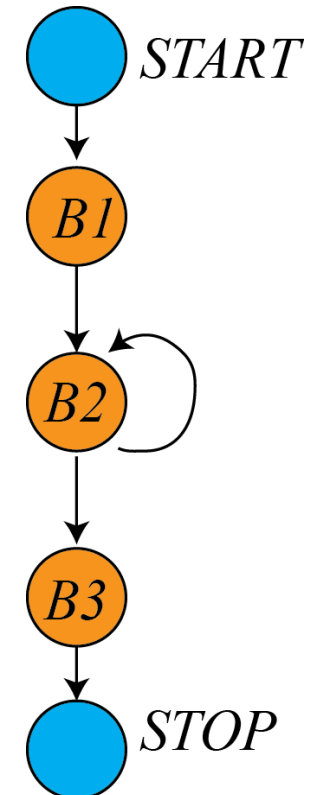
```
a = 10;  
b = 20;  
label_1: i = a * b;  
         j = i + 25;  
         t1 = 4 * a;  
         t2 = i * t1;  
         t3 = 4 * b;  
         t4 = j + 1;  
         if (i < 20) goto label_1;  
         t5 = t2 - t4;  
         t6 = t3 * t5;
```

Control Flow Graph (CFG): Definition

- Control flow graph (CFG) is a directed graph with nodes and edges:
- Nodes:**
 - Basic blocks (grouping of sequential operations without branching)
 - Two unique nodes START node and a STOP node is defined in the CFG
 - There exist directed paths from START to all the nodes
 - There exist a path from any node to the STOP node
- Edges:** represents flow between basic blocks and decision nodes



- We assume no information about data values while creating CFG
- An edge in the CFG indicates that the execution can take that path



Control Flow Graph (CFG): Example 1

Draw the CFG of the given code.

- Identify leader statements by using the following rules:
 - The first statement in the program is a leader
 - Any statement that is the target of a branch statement is a leader
 - Any statement that immediately follows a branch or return statement is a leader

```
a = 10;  
b = 20;  
j = 1;  
i = a*b;  
if (i < 150) {
```

B1

```
    j = i + 25;  
    t1 = 4 * a;  
    t2 = i * t1;
```

B2

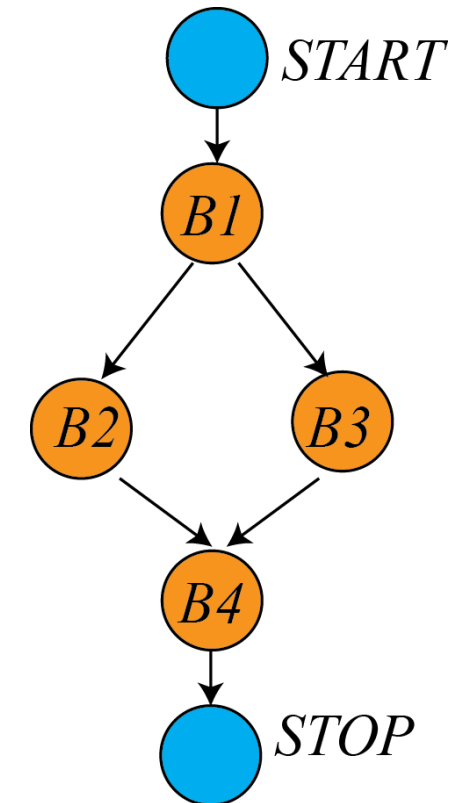
```
} else {
```

```
    t1 = 4 * b;  
    t2 = j + t1;  
    j++;
```

B3

```
}  
t3 = 4 * b;  
t4 = j + 1;
```

B4



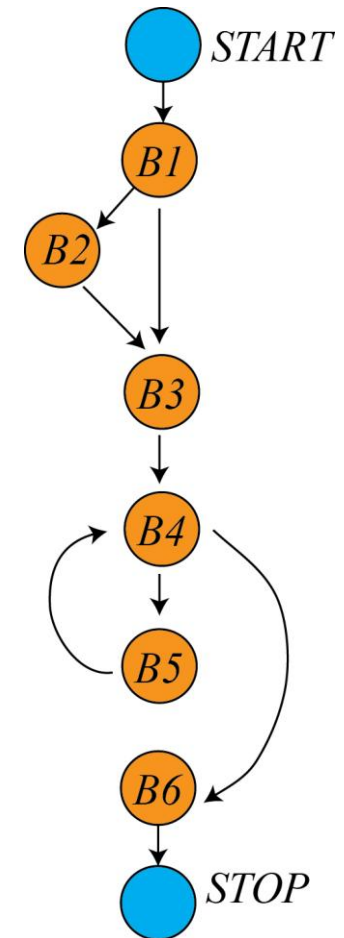
Control Flow Graph (CFG): Example 2

Draw the CFG of the given code.

- Identify leader statements by using the following rules:
 - The first statement in the program is a leader
 - Any statement that is the target of a branch statement is a leader
 - Any statement that immediately follows a branch or return statement is a leader

```
a = 10;  
b = 20;  
j = 1;  
i = a*b;  
if (i == 25) {  
    j = i + 25;  
}  
t1 = j;  
t2 = t1 + 5;  
while (a < 15) {  
    a++;  
    t3 = a - t1 + t2;  
}  
t4 = t1 + t2;  
t5 = t2 + t3;
```

B1
B2
B3
B4
B5
B6

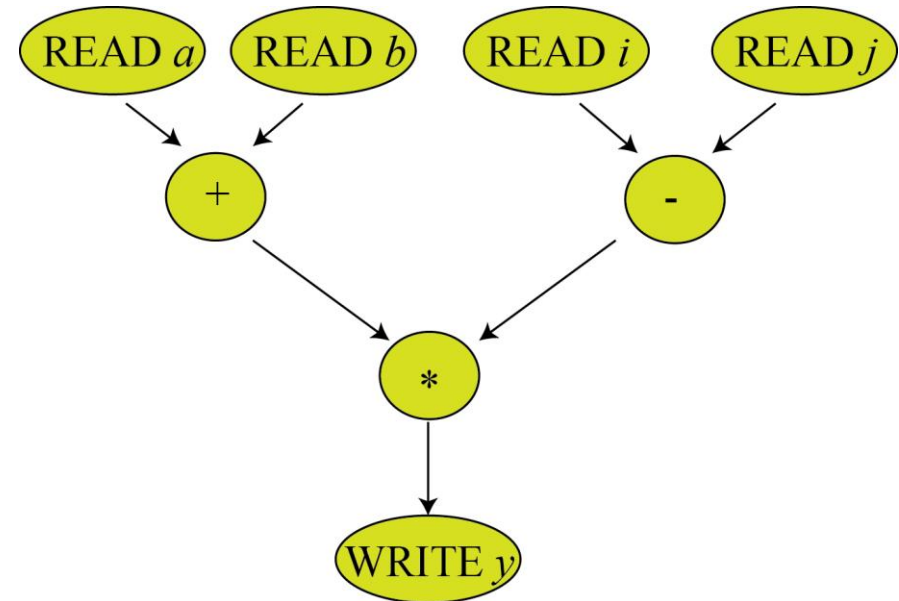


Data Flow Graph (DFG)

- Data Flow Graph (DFG) models the flow of data within a program using directed graph
 - Vertices (nodes): correspond to a computation or operation, such as addition, multiplication, or a function call
 - Edges: direction indicates the producer-consumer relationship (i.e., the result of one operation is used as input to another).
- The DFG focuses solely on the data dependencies

- Allows discovering parallelism
- Allows mapping hardware to computation
- Can be used in optimization (by manipulation of DFG)

`y = (a + b) * (i - j);`



Single Static Assignment (SSA)

- Single Static Assignment (SSA) form is often used in DFG
- This allows for easier analysis and optimization of programs

- What is SSA?
 - Each variable is assigned exactly once and every variable is defined before it is used
 - Whenever a variable is assigned another value, a new version of that variable is created

- How to derive SSA?
 - Replace the target of each assignment with a new variable
 - Replace each use of a variable with the "version" of the variable reaching that point.

```
x = read();  
y = x + 6;  
x = x + 7;  
z = x - y;
```

```
x1 = read();  
y1 = x1 + 6;  
x2 = x1 + 7;  
z1 = x2 - y1;
```

SSA: phi (Φ) function

```
x = read();  
if (x > 0) {  
    y = 5;  
} else {  
    y = 10;  
}  
x = y;
```

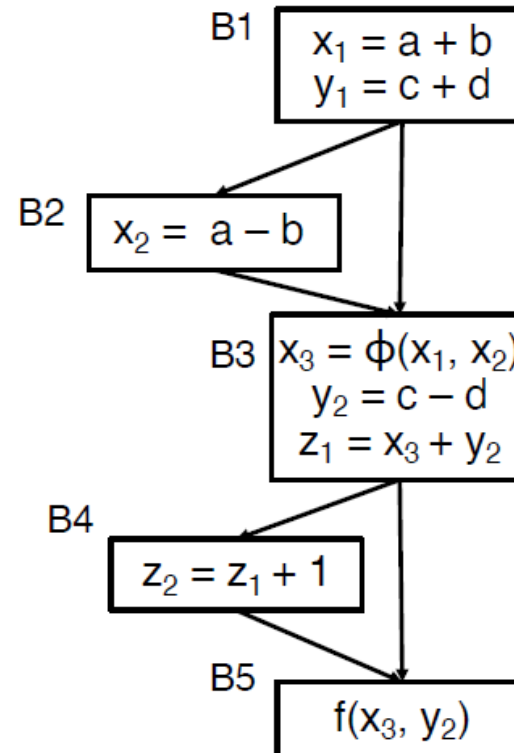
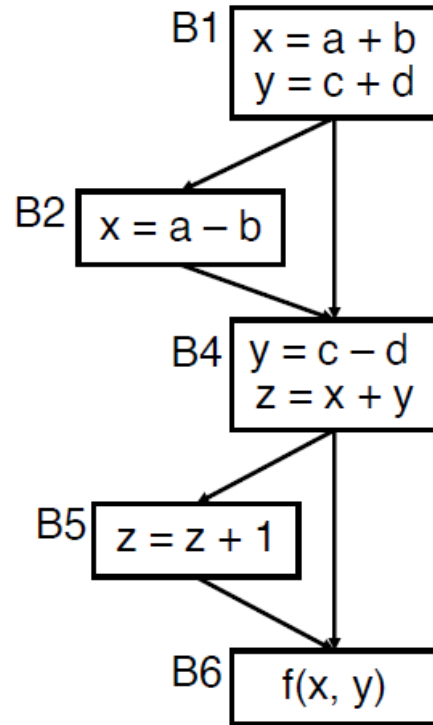
```
x1 = read();  
if (x1 > 0) {  
    y1 = 5;  
} else {  
    y2 = 10;  
}  
x2 = ???;
```

```
x1 = read();  
if (x1 > 0) {  
    y1 = 5;  
} else {  
    y2 = 10;  
}  
y3 = phi(y1, y2);  
x2 = y3;
```

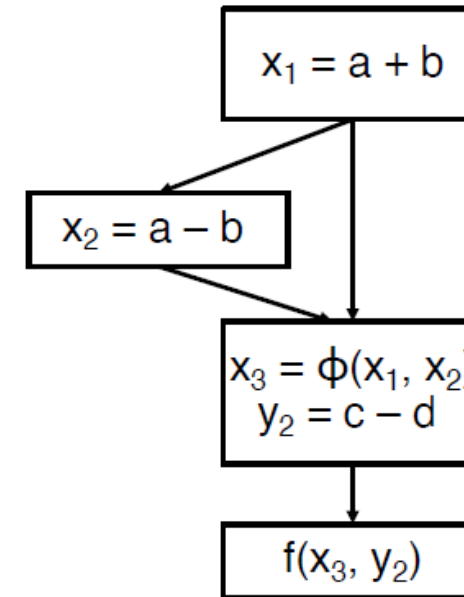
- In the case of control flow merges (such as after an if-else block), SSA introduces a special phi function (Φ) to select the correct version of the variable based on the path taken.
- Φ functions allow DFG to handle control flow in a structured manner, making it suitable for a wide range of optimizations
- Φ functions have the semantics of a multiplexer, copying the correct definition
- Multiplexers are typically used to implement Φ functions in the DFG
 - Inputs to the MUX is the input given to the Φ function
 - Select to the MUX is the condition or state that chooses the branch

Using SSA for Optimization

- Derive SSA



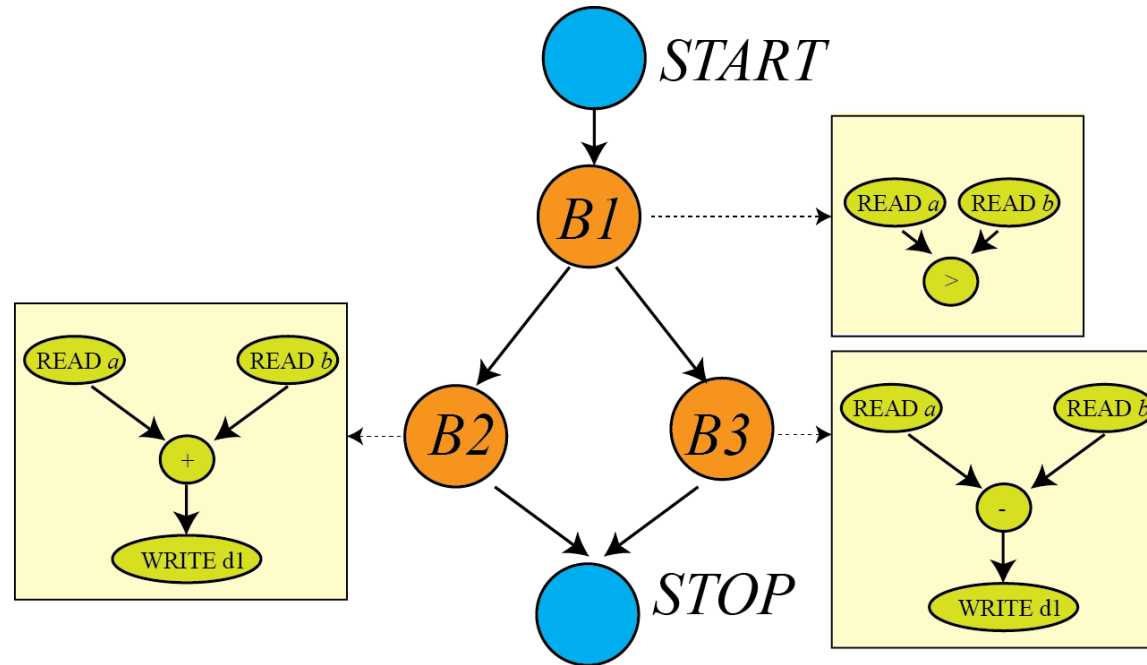
- Iteratively remove unused definitions:



- Remove y_1 , z_2 (and B4) and then remove z_1

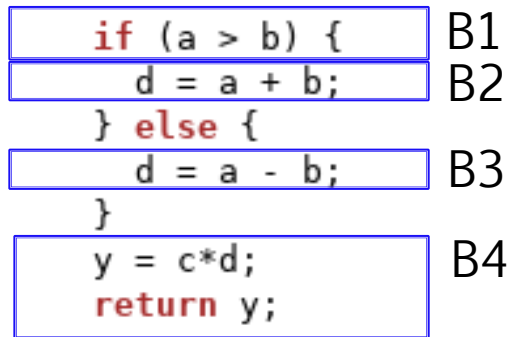
CDFG: Combine CFG and DFG

```
if (a > b) { B1  
    d = a + b; B2  
} else {  
    d = a - b; B3  
}
```

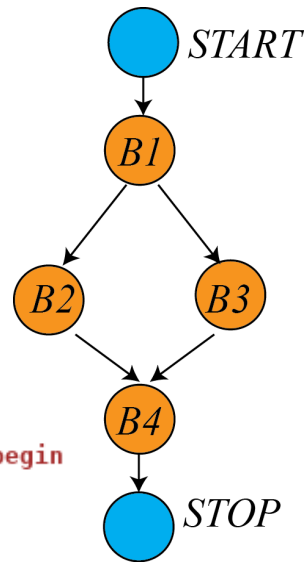


Control Flow Graph (CFG): Example 2

Draw the CFG of the given code.



```
always @(posedge clk or posedge rst) begin
  if (rst) begin
    d <= 32'b0;
    y <= 32'b0;
  end else begin
    case (state)
      B2: d <= a + b;
      B3: d <= a - b;
      B4: y <= c*d;
    endcase
  end
end
```



```
// State encoding
localparam B0 = 3'b000, // Initial state
           B1 = 3'b001, // Compare a and b
           B2 = 3'b010, // Addition state
           B3 = 3'b011, // Subtraction state
           B4 = 3'b100; // Multiplication state

always @(posedge clk or posedge rst) begin
  if (rst) begin
    state <= B0; // Reset to initial state
  end
  else begin
    case (state)
      B0: state <= B1;
      B1: begin
        // Compare a and b
        if (a > b) begin
          state <= B2; // Go to addition state if a > b
        end else begin
          state <= B3; // Go to subtraction state if a <= b
        end
      end
      B2: state <= B4; // After computing c, go to B4
      B3: state <= B4; // After computing c, go to B4
      B4: state <= B0; // After computing, go back to initial
      default: state <= B0; // Default state is initial
    endcase
  end
end
```

References

- Lavagno, L., Markov, I. L., Martin, G., & Scheffer, L. K. (Eds.). (2017). *Electronic design automation for IC system design, verification, and testing*. CRC Press.
- Micheli, G. D. (1994). Synthesis and optimization of digital circuits. McGraw-Hill Higher Education.
- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.